Dimension-Adaptive 2D Systolic Array with Workload-Aware Active Region Control for Edge-AI Inference

Track: FPGA

Application Domain: Edge-AI / ML Hardware Acceleration

Platform: Digilent BASYS 3 FPGA | HDL: System Verilog | Tool: AMD/Xilinx Vivado

ABSTRACT

Edge-AI inference uses matrix multiplication again and again, especially in **DNN layers**. A fixed **2D systolic array works well when the workload matches its dimensions**. But real layers are not always the same size. Some are small, some are rectangular, and some may not match the physical PE array properly. In that case, **several Processing Elements (PEs) remain unused**, even though they are still part of the hardware.

The proposed work takes a different approach. The physical 2D systolic array stays fixed after synthesis, but the **active PE region changes according to the incoming workload**. A workload/configuration controller reads the matrix or tile dimensions and decides how much of the PE array is actually required. Smaller workloads use a smaller active region. **Larger workloads can use more PEs**, up to the available hardware limit.

Another part of the design handles irregular matrix dimensions. Padding and tiling are used when the workload does not directly fit the selected processing dimensions. For example, a valid multiplication such as **A (5×7) × B (7×4)** can be mapped to hardware using padded dimensions such as **5×8 and 8×4**, where the added elements are zero. The final logical result remains 5×4. A skew buffer is then used to align the operands properly for systolic timing.

The design will be implemented in System Verilog on a Digilent BASYS 3 **FPGA** using **AMD/Xilinx Vivado**. A **small DNN inference worklo**ad will be used for demonstration, and the **adaptive architecture** will be compared against a **conventional fixed-array baseline**.

MAIN IDEA

Overview

The main idea is actually simple.

Keep the hardware fixed. Change how much of it is used.

A **conventional** systolic array is designed with a **fixed number of PEs**. That is fine for a workload that matches the array. But when the workload becomes smaller, the same large array is still present. A lot of those PEs are not doing useful computation.

In our approach, **the physical PE array is selected during synthesis** based on the **available resources of BASYS 3**. After synthesis, its size does not change. At runtime, only the **active region** changes.

So, the basic flow is:

Fixed Physical PE Array + Workload-Aware Active Region → Better PE Utilization

The controller receives the workload dimensions **M, N and K**, or the dimensions of the current tile. It then determines the required active region. The remaining PEs stay physically present, but they are excluded from the useful MAC computation for that workload.

This distinction is important. We are not claiming that unused PEs are physically removed or automatically power-gated. The project will measure what difference the active-region approach actually makes.

Implementation Details

The accelerator performs general matrix multiplication:

A(M×K) × B(K×N) = C(M×N)

The design does not assume square matrices. Rectangular workloads are part of the problem itself.

Padding and Tiling

Not every workload will fit the physical array directly.

For example:

A (5×7) × B (7×4) = C (5×4)

If the selected hardware works with an 8-wide processing dimension, the matrices can be padded:

A (5×7) → A (5×8)
B (7×4) → B (8×4)

The additional values are zero, so they do not change the mathematical result. After computation, the required **5×4** output is retained.

This is where padding helps. It does not change the actual problem. It only makes the data fit the selected hardware shape.

For workloads larger than the available PE array, tiling is used. The large matrix operation is divided into smaller tiles, processed one after another, and then combined to produce the required output.

Skew Buffer

Padding and skewing do different jobs.

Padding deals with dimensions.

The skew buffer deals with timing.

Operands have to reach the correct PEs at the correct clock cycles. The skew buffer introduces the required delays so that the systolic computation proceeds in the intended wave-like pattern.

Weight-Stationary Dataflow

The design uses a weight-stationary approach. Weights are loaded into the active PEs and remain there while activations move through the array. Each PE performs multiply-accumulate operations and passes the required data onward.

This reduces unnecessary movement of stored weights and fits naturally with the proposed PE-region control.

Workload / Configuration Controller

The controller acts as the decision-making part of the architecture.

It receives the current workload information, determines the active PE dimensions, manages loading and computation, and handles tiling when required. The final RTL implementation can use an FSM for these control stages.

The important thing is that the **bitstream remains the same** even when the active region changes from one workload to another.

System-Level Architecture

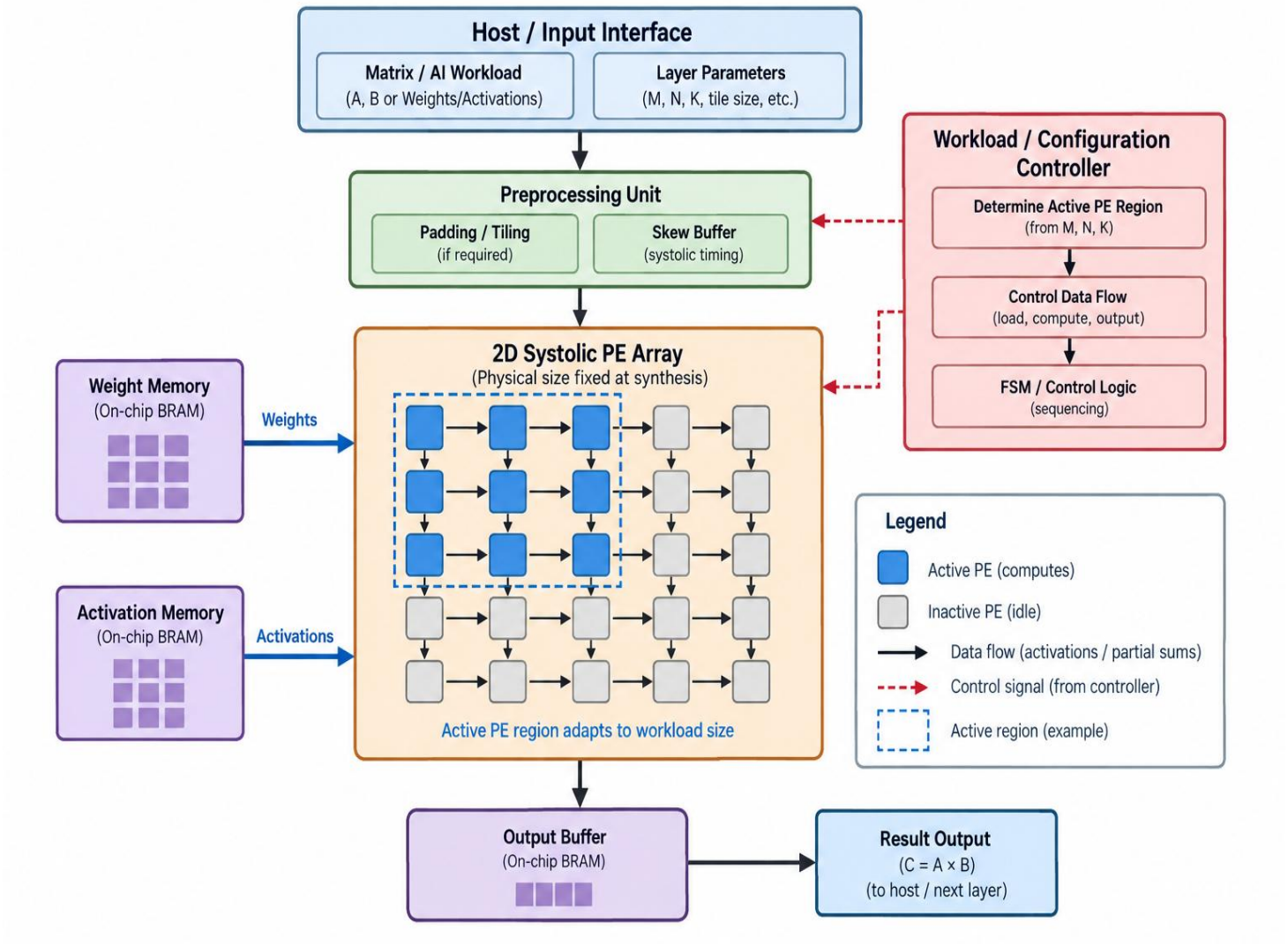


Figure 1. System-level architecture. The physical PE array is fixed after synthesis; only the active region changes with the workload.

The target platform is the Digilent BASYS 3 (Xilinx Artix-7). The HDL is System Verilog, and AMD/Xilinx Vivado will be used for synthesis, place-and-route and reporting. That part is fixed. The exact PE count is not, yet.

APPLICATION

The first application will be a small **DNN inference workload** on the BASYS 3 FPGA.

Dense or fully connected layers naturally involve matrix or matrix-vector operations. Convolutional layers can also be transformed into **GEMM-style operations**. This makes them suitable for the proposed systolic architecture.

The interesting part is not only performing the multiplication. **We want to observe what happens when different layers require different amounts of computation.**

For example, one layer may need a relatively small active region, while another may require most of the available array. The hardware remains the same, but the computation region changes.

Potential / Future Applications

The same architecture could later be extended toward:

- CNN and DNN inference on small embedded FPGAs
- DSP and signal-processing workloads
- Matrix-heavy scientific or industrial workloads
- Other Edge-AI workloads with changing matrix dimensions
- Workloads containing rectangular or irregular GEMM operations

The main requirement is that the workload should fit within the memory and I/O limitations of the target FPGA.

PROPOSED SOLUTION FLOW

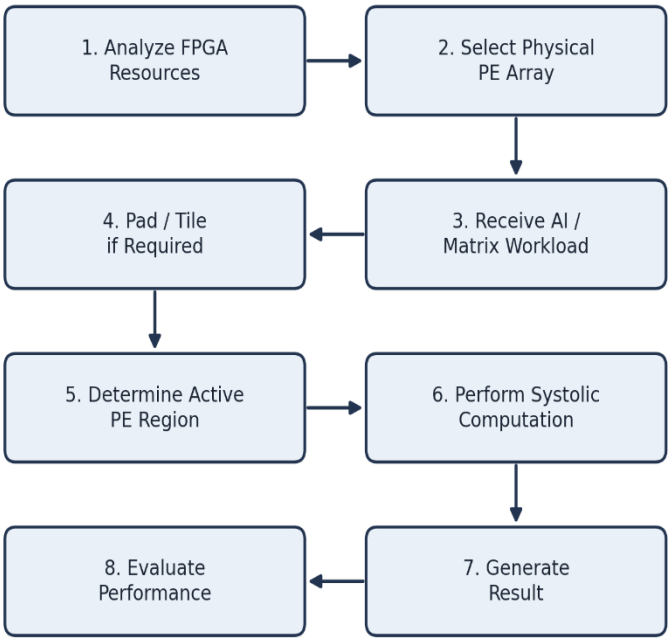


Figure 2. Proposed solution flow. Synthesis chooses the physical PE array; runtime selects only the active region needed for the current workload.

Evaluation plan

The project will compare the **adaptive design with a conventional fixed-array baseline on BASYS 3**. We will use **Vivado reports together with on-board tests.** The comparison is meant to show where the adaptive approach helps, and where it does not. We will look at PE utilization for square and rectangular workloads, **FPGA resource usage, latency and throughput, maximum operating frequency and timing closure, estimated dynamic power, and controller overhead**. This is important because a lower active-PE count does not automatically prove lower power. The measurements have to show what actually changes on the FPGA.

- PE utilization (active PEs versus physical PEs) for several square and rectangular workloads
- FPGA resources: LUT, FF, DSP and BRAM usage
- Latency, throughput, and maximum operating frequency / timing closure
- Estimated dynamic power from Vivado
- Controller / control overhead

The numbers will come after measurement. Not before. This proposal does not assume a speedup, power reduction, or resource saving without evidence.

VALUE ADD

Key Benefits

1. **Adaptive PE utilization.**
   The computation region changes with the workload instead of always using the entire physical grid.
2. **Support for irregular workloads.**
   The design does not depend on square matrices. Rectangular GEMM can be handled using padding and tiling.
3. **Better use of available FPGA resources.**
   A large physical array can support different workload sizes without requiring a different bitstream for every case.
4. **Clear separation of operations.**
   Padding handles dimensions, tiling handles larger workloads, and skew buffering handles systolic timing. Each part has a specific job.
5. **Edge-AI focused implementation.**
   The project is not only a theoretical architecture. It will be implemented and tested on a real BASYS 3 FPGA.
6. **Experimental comparison.**
   The proposed design will be compared with a conventional fixed-array baseline using PE utilization, LUT/FF/DSP/BRAM usage, latency, throughput, timing and estimated dynamic power.

Novelty / Contribution

Existing work has **already explored** systolic arrays, reconfigurable accelerators, **FPGA-based DNN acceleration and workload-dependent resource allocation**. The proposed work does not claim to invent those concepts from zero.

The contribution is in how these ideas are brought together into one practical FPGA implementation:

1. **Synthesis-time parameterized physical PE-array sizing**
2. **Runtime workload-aware active-region control**
3. **Padding and tiling for varying and irregular GEMM dimensions**
4. **Separate skew buffering for systolic timing**
5. **Evaluation of utilization, latency, resource usage, timing and power on BASYS 3**

The project therefore focuses on a practical question:

Can a fixed systolic hardware fabric be used more efficiently by adapting only the active computation region to the workload?

That is what the implementation and experimental results will determine.

REFERENCES

[1] H. T. Kung, “Why systolic architectures?” Computer, vol. 15, no. 1, pp. 37–46, Jan. 1982.

[2] N. P. Jouppi et al., “In-datacenter performance analysis of a Tensor Processing Unit,” in Proc. 44th Annu. Int. Symp. Computer Architecture (ISCA), 2017, pp. 1–12.

[3] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, “Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks,” IEEE J. Solid-State Circuits, vol. 52, no. 1, pp. 127–138, Jan. 2017.

[4] X. Wei et al., “Automated systolic array architecture synthesis for high throughput CNN inference on FPGAs,” in Proc. 54th ACM/EDAC/IEEE Design Automation Conf. (DAC), 2017.

[5] C. Zhang, P. Li, G. Sun, Y. Guan, B. Xiao, and J. Cong, “Optimizing FPGA-based accelerator design for deep convolutional neural networks,” in Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays (FPGA), 2015, pp. 161–170.

[6] Diligent, Basys 3 FPGA Board Reference Manual. Pullman, WA, USA: Diligent Inc.

[7] X. Zhang, J. Wang, C. Zhu, Y. Lin, J. Xiong, W.-M. Hwu, and D. Chen, “DNNBuilder: An Automated Tool for Building High-Performance DNN Hardware Accelerators for FPGAs,” in Proc. IEEE/ACM Int. Conf. Computer-Aided Design (ICCAD), 2018.

[8] K. Guo et al., “Angel-Eye: A Complete Design Flow for Mapping CNN Onto Embedded FPGA,” IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems, vol. 37, no. 1, pp. 35–47, Jan. 2018.

[9] Y. Umuroglu, N. J. Fraser, G. Gambardella, M. Blott, P. Leong, M. Jahre, and K. Vissers, “FINN: A Framework for Fast, Scalable Binarized Neural Network Inference,” in Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays (FPGA), 2017, pp. 65–74.

[10] A. Shawahna, S. M. Sait, and A. El-Maleh, “FPGA-Based Accelerators of Deep Learning Networks for Learning and Classification: A Review,” IEEE Access, vol. 7, pp. 7823–7859, 2019.